Variable ordering / bounded MCTS

Search mechanics · 1 of 2

The objective is the fastest completed ordering found within a budget. symGCAD implements Monte Carlo tree search (MCTS) over variable-elimination prefixes. Exact solves measure candidates; heuristic scores guide where to spend the next trial. This search supplies no certificate that its chosen ordering is globally optimal.

01 · Seed
Given order + inexpensive polynomial heuristics

02 · Explore
UCT selects a prefix; a rollout fills the remaining order

03 · Measure
Bounded exact solves; promising timeouts get longer trials

04 · Select
Fastest completed candidate; otherwise a restart suggestion

Why order matters

Each projection eliminates a variable and forms discriminants, leading coefficients and pairwise resultants. An unfortunate early elimination can increase degrees, term counts and coefficient sizes dramatically, making every later step expensive. Lifting cost also depends on the resulting root boundaries and number of cells. Consequently, the final choice uses completed solve time, including lifting.

The tree and its finite depth

A node represents an elimination prefix; an edge chooses the next legal variable. With n variables, a complete path has depth n. Variable groups constrain the legal branches. An explicit fixed order bypasses MCTS, and a homogeneous scale anchor must retain its required position.

Public orders are written in **lifting order**: an order `[x, y, z]` means eliminate z, then y, then x. The tree uses this elimination direction internally. For example, `[z, y]` fixes the first two eliminations, while a rollout fills in the remaining variables.

The implementation has **no separate depth-k lookahead setting**. It does not stop after k eliminations, choose one variable and restart the search at the next level. Its limits are elapsed time and scheduler iterations; each evaluated candidate is a complete ordering, although its algebra worker can time out partway through.

Inexpensive starting orders

The initial portfolio contains the given order and deduplicated MODS-, T1-, Brown- and sparsity-based rankings. These seeds use the original input features, avoiding a full projection for each seed. Expansion and rollout priorities likewise use a fixed input ranking; they are not recomputed from every projected polynomial set.

Selection, rollout and feedback

Unexpanded legal choices are tried in heuristic priority order. Once all children exist, UCT combines mean reward with an exploration bonus and a small, decaying preference for the heuristic ranking. The default score for a visited child is

$$Q_i + \sqrt{2}\sqrt{\frac{\ln N}{n_i}} + \frac{0.1}{(r_i+1)(n_i+1)}$$

Here Q is mean reward, N the parent visit count, n the child visit count and r its zero-based heuristic rank. Unvisited children take priority. A rollout follows the cheapest-ranked legal choice 80% of the time; otherwise it samples a legal choice uniformly. Seed 0 is the default.

Completed solves earn the largest rewards, with faster solves scoring higher. A timed-out global-projection trial can earn a smaller reward from observed factor, term, degree and coefficient-size costs. These observations predict difficulty; they do not estimate how much of the solution set has been proved.

Successive time budgets

Fresh orders start with a one-second worker limit. Timed-out candidates can be promoted to ten seconds, then sixty seconds. Promotion normally follows eight fresh trials, or happens sooner when all legal orders have been seen. Every fourth promotion favors a deeper tier so longer trials are not starved.

Killable worker processes enforce deadlines even inside an expensive resultant or factorization. Each trial is clipped to the remaining search budget. The optimized engine can also abandon prefixes with extreme observed term growth and avoid repeating them during that search campaign.

What wins

Only a completed exact solve can become `best_order`. The comparison uses measured worker wall time, with fewer cells breaking an exact timing tie. With no completion, `suggested_order` is a heuristic restart candidate. The public solve command then runs the selected order with the remaining campaign time and exports that solve's evidence.

Variable ordering / use and evidence

Budgets, reproducibility and interpretation · 2 of 2

Enable the search

Add these settings to a problem TOML, keeping its variables, polynomial expressions and domain constraints. Leave the explicit `order` field unset.

```toml
[solver]
engine = "optimized"
ordering = "mcts"
seed = 0
max_rollouts = 256
search_time_secs = 300
checkpoint_dir = "artifacts/checkpoints"

[limits]
wall_time_secs = 3600
memory_mib = 4096
workers = 1
```

`symgcad order problem.toml` runs the bounded search and reports its history. `symgcad solve problem.toml` includes the search, then solves the selected ordering. `symgcad verify result.json` checks the exported decomposition separately. MCTS is opt-in; the default ordering is `given`.

How the limits interact

Control	Effective behavior
Search wall time	At most min(300 s, 20% of campaign); an explicit search limit can lower it.
`max_rollouts`	256 scheduler iterations by default; promotions and revisits also consume iterations.
Candidate tiers	1 s ⟶ 10 s ⟶ 60 s, clipped to remaining search time.
Final solve	Receives the remaining campaign wall budget after search.

A one-hour campaign therefore allows up to five minutes for search; a 300-second campaign allows up to sixty seconds. The iteration ceiling does not promise 256 distinct exact solves. Process launch and scheduling can add small timing overheads.

Reuse and restart

Compatible trials share completed exact algebra and projection-prefix caches. An atomic journal preserves trial history, random-generator state through replay, tree visits and promotion state. Context checks bind the problem, domains, groups, solver settings and executable identity. Restored completion claims must pass a fresh bounded solve before they can win; the journal never substitutes for proof.

Soundness and the meaning of “best”

Choosing or abandoning an ordering changes search effort only. It cannot delete a solution region, omit an ordinary $F = 0$ sheet or justify a projection reduction. The selected final solve does not use the search’s heuristic prefix-abandonment rule. Its complete status still requires exhaustive algebraic construction, with an independently checked proof required for benchmark completion credit.

“Best” means the fastest completed candidate observed under the recorded limits, engine and cache conditions. It is not an exhaustive comparison of all permutations. A favorable partial score, a timeout or an old journal entry cannot establish a solved result. If the final solve also fails to finish, the problem remains incomplete.

What the measurements establish

One controlled experiment tested the optional `search_local_progress = true` policy on a five-dimensional positive Gluza30 base using the local engine. The option uses polynomial inventory and refinement growth to rank timed-out candidates for promotion. It is disabled by default and changes neither UCT rewards nor the completed-winner rule.

Promotion policy	Verified	Median solve
Existing scoring	0 / 3	313.62 s stop
Local inventory scoring	3 / 3	65.10 s

These paired fresh-process runs used the same release executable, seed 0, cold checkpoints, a sixty-second search budget, one CPU and 4 GiB per worker. Enabled runs produced 15,389 cells; median separate verification took 2.50 seconds. The reported solve time includes search. The unfinished controls do not yield an exact speed ratio, and this experiment compares two MCTS promotion policies, not MCTS against a heuristic-only solver.

This is evidence of benefit on one reduced problem. The earlier multi-problem ordering study did not establish an overall MCTS advantage; the new result does not establish a corpus-wide improvement or explain the matched-baseline speedup. Search overhead and shared-cache effects must remain visible in comparisons.

Implementation and audit trail

`src/search.rs`: tree policy, rewards, tier scheduling and prefix abandonment.
`src/ordering.rs`: inexpensive seed portfolio and group conventions.
`src/main.rs`: process trials, budgets and final solve.
`src/search/checkpoint.rs`: bounded journal and restart validation.
`docs/USAGE.md`: configuration and command usage.
`reference/LOCAL_SEARCH_PROMOTION_BENCHMARK.md`: paired experiment, limits, source identities and retained evidence.